



Colegiul Național
Emil Racoviță Iași

Excelență în educație

DIGITAL LOGIC SIMULATION

&

CIRCUIT BUILDER

**LUCRARE PENTRU OBTINEREA ATESTATULUI
PROFESIONAL ÎN INFORMATICĂ**

Elevi: Acatrinei David-Cristian, Ene Andrei

Clasa: XII A 2025-2026

Profesor coordonator: Cerchez Emanuela

CUPRINS

I. INTRODUCERE.....	3
I.1. MOTIVAȚIA PROIECTULUI.....	3
I.2. TEMATICA PROIECTULUI	3
I.3. CONVENȚII ȘI DECIZII LUATE	4
I.3.1. DEPENDENȚE	4
I.3.2. CONVENȚII DE SCRIERE.....	4
II. STRUCTURA.....	6
II.1. NUCLEU.....	6
II.1.1. BAZĂ.....	6
II.1.2. OPȚIONAL.....	14
II.1.3. CONFIGURARE	19
II.2. APLICAȚIE	20
II.2.1. SISTEMUL DE RENDERING	20
II.2.2. SISTEMUL DE COORDONATE	20
II.2.3. COMPONENTELE SIMULATORULUI.....	21
II.2.4. LOGICA SIMULATORULUI	21
III. DETALII TEHNICE.....	22
III.1. COMPILARE.....	22
IV. UTILIZAREA APLICAȚIEI.....	23
V. BIBLIOGRAFIE.....	25

I. INTRODUCERE

I.1. MOTIVAȚIA PROIECTULUI

În lumea informaticii din prezent se observă o creștere exponențială în abstractizarea interacțiunii dintre programator și calculator. Cele mai multe produse soft sunt scrise peste biblioteci uriașe ce conferă o mare parte din funcționalitatea programului. Mai mult de atât, evoluția uriașă în domeniul Inteligenței Artificiale permite automatizarea întregului proces de Software Development.

În prim plan, această abstractizare nu este ceva negativ, permite reutilizarea funcționalității, crearea unor proiecte mai complexe, generalizarea anumitor soluții și ușurează munca programatorului. Totuși, odată cu creșterea nivelului de abstractizare, observăm creșterea numărului de produse incomplete, ineficiente, din punct de vedere al memoriei, cât și al timpului de execuție, dar și vulnerabilitățile sau bug-urile acestor produse.

Cea mai proeminentă consecință a acestei abstractizări este efectul Black Box¹, un efect pe care noi îl considerăm negativ pentru un programator și dăunător pentru un elev sau începător în domeniul informaticii.

În acest sens, propunem acest proiect drept unealtă educațională pentru oricine dorește să abordeze logica digitală la un nivel cât mai aproape de componentele fizice ale sistemului de calcul.

Acest proiect a reprezentat o oportunitate de învățare enormă în dezvoltarea produselor soft și dorim să mulțumim tuturor resurselor gratuite ce oferă explicații și ajutor elevilor și începătorilor în programare.

I.2. TEMATICA PROIECTULUI

Acest proiect este un produs soft, menit să ruleze nativ pe un sistem de operare ce admite interfață grafică. În prezent proiectul a fost testat pe sistemul de operare Microsoft Windows 10/11 64bit, pe procesoare ce folosesc arhitectura x86, dar poate fi extins pentru sistemele din familia UNIX, cu puține ajustări.

Proiectul aparține familiei de programe de tip simulator, ce prezintă utilizatorului o interfață prietenoasă pentru a elabora un circuit abstract, compus din porți logice, indicatori ale stării curentului pentru datele intrare/ieșire și conectori.

Utilizatorul este echipat cu o singură poartă logică de tip ȘI-NU (NAND) și este încurajat să folosească diferite combinații ale acestei porți pentru a modela celelalte operații binare și mai apoi circuite complexe pentru a simula funcționalitatea unui sistem de calcul complet.

¹ Black Box Effect = sistem abstract ce poate fi observat doar prin datele de intrare și ieșire, comportamentul intern nu este cunoscut de către utilizator

I.3. CONVENȚII ȘI DECIZII LUATE

I.3.1. Dependente

În spiritul minimizării nivelului de abstractizare, am ales să realizăm acest proiect folosind limbajul de programare C, respectând standardul C99². Pe lângă biblioteca standard, acest proiect se folosește de 3 biblioteci externe de tip Open-Source³: *GLFW*, *GLAD* și *stb_image* și un API⁴ de grafică pentru a accesa placa video a sistemului, *OpenGL*.

GLFW este o bibliotecă Open-Source scrisă în C, elaborată pentru mai multe sisteme de operare/medii desktop, ce oferă funcționalitate pentru deschiderea panourilor/contexte grafice și interacțiunea cu utilizatorul printr-un sistem de evenimente. Mai multe detalii despre această bibliotecă pot fi găsite la <https://www.glfw.org/>.

stb_image este o bibliotecă Open-Source scrisă în C, de către Sean Barrett, ce oferă funcționalitate pentru citirea fișierelor de tip imagine. Mai multe detalii pot fi găsite la <https://github.com/nothings/stb>.

OpenGL este o specificație API, apărută în 1992, ce descrie un sistem abstract pentru programarea componentelor grafice în 2D sau 3D. Fiind o specificație/listă de specificații, *OpenGL* NU este o bibliotecă, implementarea sistemului descris fiind responsabilitatea dezvoltatorului de plăci video. Programul propriu-zis ce implementează sistemul *OpenGL*, apare, așadar, în driverul⁵ cu care este dezvoltată placa video. Pentru a avea acces la funcționalitatea dată de driver, vom folosi o bibliotecă Open-Source numită *GLAD*, ce caută în lista de simboluri a driverului și atașează instrucțiunile la numele de funcții descrise în specificația *OpenGL*, mai multe detalii pot fi găsite la <https://github.com/Dav1dde/glad>.

I.3.2. Convenții de scriere

În această lucrare se vor regăsi secvențe de cod din proiect; pentru a stabili un model ușor de urmărit definim următoarele convenții de scriere a codului.

Tipurile de date utilizate sunt definite de biblioteca standard `<inttypes.h>` și redefinite în acest proiect cu următoarea structură: prima literă reprezintă tipul general al datelor [i/u/f], numere întregi cu semn, întregi fără semn, respectiv numere reale, următoarele cifre reprezintă numărul de biți necesari reprezentării numărului în memorie [8/16/32/64/128]. Prin urmare, avem următoarele tipuri de date:

<code>typedef uint8_t</code>	<code>u8;</code>
<code>typedef uint16_t</code>	<code>u16;</code>
<code>typedef uint32_t</code>	<code>u32;</code>
<code>typedef uint64_t</code>	<code>u64;</code>
<code>typedef int8_t</code>	<code>i8;</code>
<code>typedef int16_t</code>	<code>i16;</code>
<code>typedef int32_t</code>	<code>i32;</code>
<code>typedef int64_t</code>	<code>i64;</code>
<code>typedef float</code>	<code>f32;</code>
<code>typedef double</code>	<code>f64;</code>
<code>typedef long double</code>	<code>f128;</code>

² În secțiunea [II.1.1. Bază](#) vom folosi o extensie a compilatorului GCC ce nu este prezentă în standardul C99

³ Open-Source = model de elaborare a produselor soft ce se bazează pe publicarea implementării proiectului

⁴ API (Application Programming Interface) = interfață ce permite comunicarea între programe, utilizată când implementarea unui program nu este publică, dar oferă funcționalitate pentru a interacționa cu acesta

⁵ Driver = program ce acționează drept interfață între o componentă hardware și una software

Pe lângă acestea, definim următoarele 2 tipuri de date pentru a clarifica intenția utilizării unor variabile.

```
typedef u8          byte;  
typedef size_t      usize;
```

Cu toate acestea, utilizăm în continuare și tipul `char` când lucrăm cu șiruri de caractere.

În continuare definim convenții de scriere a funcțiilor și structurile de date definite de programator.

În secțiunea [II. Structura](#) vor fi prezentate în amănunt deciziile legate de arhitectura proiectului, dar, în principiu, încercăm să separăm funcționalitatea în module. Așadar, fiecare nume de funcție va avea următoarea structură: primul cuvânt reprezintă modulul din care face parte funcția, urmat de „_”, al doilea cuvânt reprezintă obiectul/componenta pe care lucrează funcția, al treilea cuvânt reprezintă acțiunea pe care o îndeplinește funcția. După aceste 3 se pot adăuga alți identificatori/discriminatori ai funcției, astfel încât acțiunea pe care o îndeplinește să fie cât mai clară. Câteva exemple de funcții pe care le vom regăsi în cod sunt:

```
render_texture_create_image(...);  
render_texture_create_file(...);  
render_texture_destroy(...);
```

Structurile de date definite de programator sunt formate din 1 sau mai multe cuvinte, scrise cu prima literă majusculă, fără separatori între ele. Spre exemplu:

```
struct WindowSpecification { ... };  
struct DataFileNode { ... };  
struct SpriteRenderer { ... };
```

Proiectul se folosește des de constante definite la nivelul preprocesorului sau macrouri, acestea sunt formate din cuvinte separate prin „_”, scrise cu majuscule și cu prefixul „ENN_”, pentru a preveni eventualele redefiniri ale unor constante cu același nume. Spre exemplu:

```
#define ENN_PLATFORM  
#define ENN_DEBUG_MODE  
#define ENN_VECTOR_INIT_CAPACITY
```

II. STRUCTURA

Arhitectura unui produs soft este una dintre cele mai importante părți ale elaborării unei aplicații. Pentru acest proiect am ales o structură App-Core sau Aplicație-Nucleu. În directorul *Dev* se regăsesc 3 subdirectoare: *App*, *Core* și *Dep*.

Core conține funcționalitate ce poate fi reutilizată pentru mai multe proiecte și reprezintă nucleul proiectului. Secțiunea [II.1. Nucleu](#) prezintă în amănunt această funcționalitate.

App conține codul specific pentru această aplicație, acesta se folosește de *Core* pentru a elabora funcționalitatea unică a acestui proiect. Secțiunea [II.2. Aplicație](#) prezintă mai multe despre detaliile acestui proiect.

Dep conține dependențele proiectului descrise în secțiunea [I.3.1. Dependente](#), acestea sunt incluse automat în *Core*.

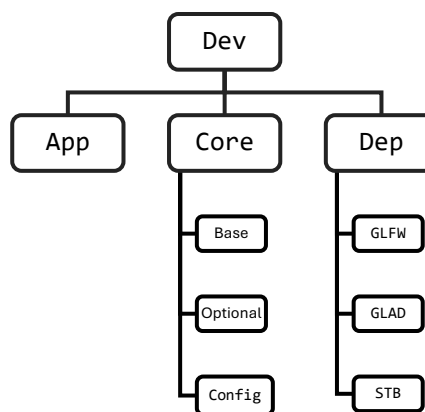


Figura II.1. Structura directorului Dev

II.1. NUCLEU

Nucleul prezintă un sistem complex de utilitare și este la rândul lui împărțit în 2 secțiuni.

II.1.1. Bază

Baza este cea mai amplă parte a nucleului și include utilitare esențiale pentru un program cu o complexitate crescută. Baza este împărțită în mai multe module cu diferite roluri, acestea sunt: Detectarea platformei, Macrouri ajutătoare, Logging și Debug, Structuri generice, Structuri matematice, Subprograme paralele, Directoare și fișiere, Serializare Text.

Detectarea platformei

Modulul de detectare a platformei se folosește de macroui predefinite pentru a verifica sistemul de operare, compilatorul și versiunea standardului. Aceste informații sunt furnizate către programator prin 3 macroui:

```
#define ENN_PLATFORM      ENN_WINDOWS | ENN_LINUX | ENN_APPLE

#define ENN_COMPILER      ENN_GCC | ENN_CLANG | ENN_MSVC

#define ENN_STD_VERSION  ENN_C89 | ENN_C99 | ENN_C11 |
                          ENN_C17 | ENN_C23 | ENN_CPP
```

În cazul în care modulul nu poate detecta una dintre aceste informații, macroul va avea valoarea `ENN_UNKNOWN` și programul nu se va compila.

Macrouri ajutătoare

Modulul de macrouri ajutătoare definește următoarele

```
#define ENNDEF_PRIVATE  
#define ENNDEF_PUBLIC
```

Funcțiile cu implementarea publică (codul este vizibil în același fișier) trebuie declarate folosind ENNDEF_PUBLIC, iar cele cu o implementare privată (codul se află în alt fișier) trebuie declarate folosind ENNDEF_PRIVATE.

```
#define TO_STR(_X)  
#define JOIN(_X, _Y)
```

Aceste macrouri îndeplinesc funcții similare cu șirurile de caracter, TO_STR transformă orice primește într-un șir de caractere prin a adăuga “ înaintea de tokenul primit. JOIN concatenează 2 tokenuri primite, fără a le transforma în șiruri de caractere.

```
#define GET_NUM_ARGS(...)  
#define HAS_ONE_ARG(...)
```

Aceste macrouri folosesc o proprietate a macrouilor cu un număr variabil de argumente pentru a detecta numărul de argumente cu care este apelat macroul. GET_NUM_ARGS acceptă un număr variabil de argumente ($0 \leq N \leq 63$) și înlocuiește apelul său cu numărul de argumente. În mod similar HAS_ONE_ARG, înlocuiește apelul său cu 1 sau 0 dacă este apelat cu un singur argument.

```
printf(TO_STR(HAS_ONE_ARG(1))); // Va afișa 1  
printf(TO_STR(HAS_ONE_ARG())); // Va afișa 0  
printf(TO_STR(HAS_ONE_ARG(1, 2, 3))); // Va afișa 0  
  
printf(TO_STR(GET_NUM_ARGS(1, 2, 3))); // Va afișa 3  
printf(TO_STR(GET_NUM_ARGS())); // Va afișa 0
```

Logging și Debug

Sistemele de debug sunt notorii în lumea informaticii pentru dificultatea lor, mai ales în limbajele de programare cu un nivel de abstractizare scăzut, cum este C. Biblioteca noastră se folosește de un sistem de macrouri pentru a preveni cele mai comune erori.

Mai întâi, biblioteca inițializează 2 buffere⁶ și o stivă. Acestea sunt:

`_internal_general_logs`, ce memorează un jurnal general al activității desfășurate de aplicație, acesta este populat prin apelul următoarelor macrouri: `LOG(...)`, `LOG_WARN(...)`, `LOG_ERR(...)`, `ASSERT(...)`. Formatul acceptat de aceste macrouri este echivalent cu cel folosit de funcțiile de afișare din biblioteca standard `<stdio.h>`.

`_internal_memory_logs`, ce, prin redefinirea funcțiilor de manipulare a memoriei dinamice `malloc`, `calloc`, `realloc` și `free`, ține evidența alocărilor și dealocărilor memoriei dinamice, calculează memoria totală utilizată și verifică dacă toate ariile de memorie au fost dealocate. (Doar dacă `ENN_DEBUG_MODE` este definit)

`_internal_call_stack`, ce simulează stiva de apeluri a funcțiilor. Datorită limitărilor limbajului, pentru a menține această stivă, este necesar apelul unui macro `DEBUG_TRACE(...)` la începutul oricărei funcții, ce poate fi apelat fără argumente pentru a extrage numele funcției în care a fost apelat, sau cu un singur argument, reprezentând un nume pentru funcția în care se află. Similar, macroul `DEBUG_UNTRACE()` trebuie apelat înainte ca funcția să returneze pentru a o elimina din stivă. (Doar dacă `ENN_DEBUG_MODE` este definit)

Biblioteca oferă următoarele macrouri:

`LOG(...)` ce scrie mesajul transmis, într-un mod echivalent cu funcții precum `printf()`, `fprintf()`.

`LOG_WARN(...)`, ce scrie mesajul transmis, adăugând evidența liniei și fișierului în care a fost apelat.

`LOG_ERR(...)`, ce scrie mesajul transmis, adaugă evidența liniei și fișierului și încheie complet programul.

`ASSERT(...)`, ce poate fi apelat cu 1 argument, reprezentând o expresie logică pe care o evaluează, în cazul în care expresia este falsă, programul este închis. Dacă este apelat cu mai mult de 1 argument, macroul se comportă similar dar scrie și mesajul transmis.

Biblioteca oferă macrourele echivalente `DEBUG_LOG(...)`, `DEBUG_LOG_WARN(...)`, `DEBUG_LOG_ERR(...)` și `DEBUG_ASSERT(...)`. (Doar dacă `ENN_DEBUG_MODE` este definit)

Macroul `ENN_DEBUG_MODE` stabilește comportamentul acestui modul. În timpul dezvoltării unei aplicații este recomandat ca acest macro să fie definit pentru a putea depista erorile. La lansarea aplicației acest macro poate fi nedefinit pentru a optimiza programul, în acest caz toate macrourele în afară de `LOG(...)`, `LOG_WARN(...)`, `LOG_ERR(...)`, `ASSERT(...)`, nu au niciun efect.

⁶ Buffer = Arie de memorie intermediară utilizată, de obicei, pentru a reduce apelul unor funcții sau transferul frecvent de date

După terminarea programului, bufferele sunt scrise în fișiere din directorul *Simulation/Data/Logs*. O utilizare corectă a acestui sistem de debug arată în felul următor:

main.c

```
#include "core.h"

void f(void);
void f2(void);

signed main(void) {
    DEBUG_TRACE();
    core_init();
    LOG("Programul incepe");
    void* mem = malloc(100);
    LOG("mem = %p", mem);
    f();
    core_term();
    DEBUG_UNTRACE();
    return 0;
}

void f(void) {
    DEBUG_TRACE(funcția1);
    LOG("funcția1 a fost apelată");
    f2();
    DEBUG_UNTRACE();
}

void f2(void) {
    DEBUG_TRACE();
    LOG("f2 a fost apelată");
    ASSERT(1 > 2, "1 nu este mai mare ca 2");
    DEBUG_UNTRACE();
}
```

2026-04-16-15-29-40.log

```
[15:29:40]: Programul incepe
[15:29:40]: mem = 000002050EA22920
[15:29:40]: funcția1 a fost apelată
[15:29:40]: f2 a fost apelată
[15:29:40]: Assertion Failed at App/main.c:30
    ASSERT(1 > 2);
    1 nu este mai mare ca 2

Call Stack:
    #1 App/main.c:28 f2
    #2 App/main.c:21 funcția1
    #3 App/main.c:7 main
```

2026-04-16-15-29-40.memlog

```
[15:29:40]: Alloc 128B      -> 000002050EA22920 | App/main.c:11

Total Memory Usage: 128 B, 0.125000 KiB, 0.000122 MiB
Deallocated Memory: 0 B, 0.000000 KiB, 0.000000 MiB
Remaining Memory (Leaks): 128 B, 0.125000 KiB, 0.000122 MiB

Remaining memory allocations:
128B      -> 000002050EA22920 | App/main.c:11
```

Structuri Generice

Unele dintre cele mai folositoare unelte găsite în limbajele de programare moderne sunt structurile dinamice, cum ar fi tablouri dinamice, liste înlanțuite, stiva sau coada alocată dinamic. Toate acestea sunt relativ ușor de implementat într-un limbaj precum C, însă un asemenea limbaj nu prezintă un mod de a reutiliza o implementare pentru mai multe tipuri de date, așadar același cod trebuie rescris pentru fiecare tip de date folosit. Biblioteca noastră vine cu o soluție pentru această problemă prin 2 structuri de date generice: **vector** și **deque**. Aplicația noastră se folosește des de **vector**, așadar îl vom prezenta pe acesta.

vector este o structură generică ce implementează un tablou dinamic ce poate fi folosit pentru orice tip de date. Un exemplu de utilizare a acestuia este:

```
vector(i32) v = vector_new();
vector_push_back(v, 1);
vector_push_back(v, 2);
vector_push_back(v, 3);
vector_destroy(v);
```

Implementarea internă a acestuia se bazează, ca multe alte elemente din această bibliotecă, pe macrouri. Structura internă pentru **vector** este:

```
#define vector(T)
    struct {
        T*      data;
        i32     capacity;
        i32     start;
        i32     end;
    }
```

Operațiile posibile pe această structură sunt:

```
vector_size(vec);
vector_clear(vec);
vector_destroy(vec);
vector_duplicate(src, dest);
vector_reserve(vec, count);
vector_shrink(vec);
vector_push_back(vec, elem);
vector_pop_back(vec);
vector_remove_at_address(vec, ptr);
vector_remove_at_index(vec, index);
vector_sort(vec, cmp, start, end);
vector_binary_search(vec, cmp, elem, pos, start, end);
```

`vector_sort` și `vector_binary_search` se folosesc de `cmp`, o funcție de comparare a elementelor din vector ce trebuie să accepte 2 variabile de tipul elementelor din vector și să returneze o variabilă de tip:

```
typedef enum ENN_CMP {  
    ENN_EQUAL      = 0x1,  
    ENN_BIGGER     = 0x2,  
    ENN_SMALLER    = 0x3,  
    ENN_ERR        = 0x0  
} ENN_CMP;
```

`vector_sort` va interschimba elemente dacă `cmp` returnează `ENN_SMALLER`.

Pe lângă aceste structuri, biblioteca oferă și o modalitate de a interschimba 2 variabile, indiferent de tipul lor:

```
#define swap(_X, _Y)  
do {  
    DEBUG_TRACE(swap);  
    DEBUG_ASSERT((sizeof (_X)) == (sizeof (_Y)));  
    byte __swap_buff[(sizeof (_X))];  
    memcpy(__swap_buff, &(_Y), (sizeof (_X)));  
    memcpy(&(_Y), &(_X), (sizeof (_X)));  
    memcpy(&(_X), __swap_buff, (sizeof (_X)));  
    DEBUG_UNTRACE();  
} while (0)
```

Structuri matematice

Acest modul definește structuri de vectori și matrici matematice pentru fiecare tip de date. Spre exemplu:

```
typedef struct { f32 x, y, z, w; }    f32vec4;
typedef f32                          f32mat4[16];
```

Subprograme paralele

Subprogramele paralele sau threadurile sunt un subcapitol complex al informaticii. Deoarece această aplicație nu se folosește de ele, nu le vom prezenta în această lucrare.

Directoare și fișiere

Acest modul conține câteva utilitare folositoare pentru lucrul cu directoare și citirea integrală a unui fișier sau scrierea unui.

```
i32      directory_create(const char* name);
DIR*     directory_open(const char* name);
i32      directory_close(DIR* name);
DirEntry* directory_read(DIR* name);

i32 file_read_cstring(const char* filename, char** data);
i32 file_write_cstring(const char* filename, char* data, i32 size);
```

Serializare text

Serializarea este procesul prin care date structurate sunt transformate într-un format pentru memorare îndelungată, din care pot fi reconstruite. Pentru a asigura integritatea datelor, acest modul se folosește de serializarea text, spre deosebire de cea binară.

Programul citește și scrie fișiere cu extensia *.enn*, structura datelor se bazează pe un arbore cu rădăcină. Fiecare nod al arborelui este format dintr-un șir de caractere, reprezentând cheia prin care este identificat și o valoare ce poate fi: un șir de caractere, o valoare întreagă, o valoare reală, o listă de noduri.

```
typedef struct DataFileNode {
    char key[ENN_DATAFILE_MAX_STRING_SIZE];
    ENN_DATAFILE_DATA_TYPES type;
    union {
        i32 int_val;
        f32 real_val;
        char string_val[ENN_DATAFILE_MAX_STRING_SIZE];
        vector<struct DataFileNode*> children;
    } data;
    struct DataFileNode* parent;
} DataFileNode;
```

Fiecare nod poate fi accesat printr-o cale reprezentată de o succesiune de chei separate prin caracterul „, | ”. Operațiile posibile pentru fiecare nod din acest format sunt put și get. Spre exemplu:

```
datafile_put_i32(&df, "elem1|elem2|elem3", 22);
datafile_put_f32(&df, "elem1|elem4|elem4", 22.1);
datafile_put_cstring(&df, "elem1|elem2|elem5", "22");
```

Aceste operații vor rezulta, în urma scrierii, într-un fișier ce conține:

```
"elem1" = [
  "elem2" = [
    "elem3" = 22,
    "elem5" = "22"
  ],
  "elem4" = [
    "elem4" = 22.100000
  ]
]
```

Se observă faptul că cheile fiecărui nod nu trebuie să fie unice, dar căile complete către fiecare nod sunt, dacă două elemente sunt scrise cu aceeași cale programul va suprascrie ultima valoare specificată.

Funcțiile ce prelucrează acest format de serializare sunt:

```
void datafile_create(DataFile* df);
void datafile_destroy(DataFile* df);

void datafile_read(DataFile* df, const char* filepath);
void datafile_write(DataFile* df, const char* filepath);

void datafile_put_i32(DataFile* df, char* keypath, i32 val);
void datafile_put_f32(DataFile* df, char* keypath, f32 val);
void datafile_put_cstring(DataFile* df, char* keypath, const char* val);

i32 datafile_get_i32(DataFile* df, char* keypath);
f32 datafile_get_f32(DataFile* df, char* keypath);
char* datafile_get_cstring(DataFile* df, char* keypath);
```

II.1.2. Opțional

Partea opțională a nucleului include module pentru a lucra cu contexte grafice. Modulele acestei biblioteci sunt: Sistemul de Layere, Sistemul de Evenimente, Sistemul de Panouri și Utilitare Grafice.

Sistemul de Layere

Organizarea elementelor unei aplicații grafice poate fi realizată în mai multe moduri. Această bibliotecă folosește un sistem de layere sau straturi. Fiecare layer conține 5 funcții:

```
void (*init)(void);
void (*term)(void);

void (*on_event)(Event* event);
void (*on_update)(f64 dt);
void (*on_render)(void);
```

Aceste funcții reprezintă, de fapt, interacțiunea programatorului cu nucleul. Programatorul definește aceste funcții, iar nucleul le apelează în timpul execuției programului.

Pe lângă funcții, fiecare layer conține și un atribut de prioritate și dacă este activ sau nu.

```
typedef struct Layer {
    LayerID    id;
    i32        priority;
    bool        active;

    void (*init)(void);
    void (*term)(void);

    void (*on_event)(Event* event);
    void (*on_update)(f64 dt);
    void (*on_render)(void);
} Layer;
```

În funcție de prioritate și statutul de activ, fiecare layer stă unul peste altul. Funcțiile fiecăruia fiind apelate în ordinea descrescătoare a priorității lor. Evenimentele sunt, așadar, propagate în ordinea priorității către fiecare layer și pot fi oprite de către fiecare layer din a se propaga mai departe, mai multe detalii în modulul Sistemul de Evenimente.

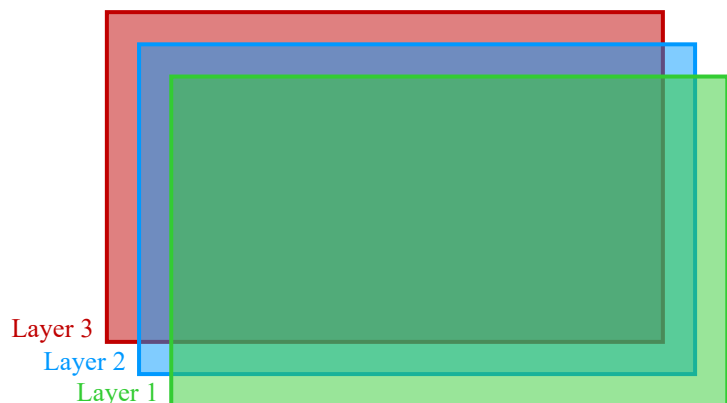


Figura II.1.2.1. Sistemul de Layere

Sistemul de Evenimente

Deoarece aplicația funcționează pe baza unei interfețe grafice, comportamentul acesteia depinde de interacțiunea utilizatorului cu dispozitivele I/O. Așadar, avem nevoie de o metodă de a primi informații despre acestea. Biblioteca *GLFW* vine cu o soluție ce se folosește de funcțiile callback⁷ pentru a transmite tastele sau butoanele apăsate de utilizator. Folosind aceste funcții, nucleul generează evenimente cu structura:

```
typedef struct Event {
    EventID      id;
    ENN_EVENT_TYPE type;
    void*        data;
    bool         handled;
} Event;
```

Fiecare eveniment este apoi transmis în ordine fiecărui layer. Deoarece evenimentele sunt transmise prin referință, fiecare layer poate modifica atributul *handled* pentru a opri propagarea către layerele cu o prioritate mai mică.

Fiecare eveniment este de un anumit timp și transmite date corespunzătoare fiecărui tip prin atributul *data*. Tipurile și datele asociate acestora sunt:

ENN_WINDOW_REGISTER_EVENT	struct { WindowID id; }
ENN_WINDOW_CLOSE_EVENT	NU TRANSMITE DATE
ENN_WINDOW_RESIZE_EVENT	struct { i32 width, height; }
ENN_WINDOW_MOVE_EVENT	struct { i32 x, y; }
ENN_INPUT_KEY_EVENT	struct { i32 key, action; }
ENN_INPUT_MOUSE_BUTTON_EVENT	struct { i32 button, action; }
ENN_INPUT_MOUSE_SCROLL_EVENT	struct { f64 offset; }
ENN_INPUT_MOUSE_MOVE_EVENT	struct { f64 x, y; }
ENN_INPUT_TEXT_EVENT	struct { u32 code; }

În plus, nucleul acceptă și evenimente definite de utilizator prin macroul *ENN_USER_DEFINED_EVENTS*.

Un exemplu de utilizare a acestui sistem de evenimente este:

```
void example_layer_on_event(Event* event) {
    switch (event -> type) {
        case ENN_INPUT_MOUSE_MOVE_EVENT: {
            struct { f64 x, y; }* data = event -> data;
            LOG("Mouse position = %lf %lf",
                data -> x, data -> y);
            break;
        }
        default: break;
    }
}
```

⁷ Funcții callback = o tehnică de programare ce se folosește de pointerii de funcții pentru a permite unei funcții să apeleze altă funcție fără ai cunoaște numele propriu-zis

Sistemul de Panouri

Pentru a crea o aplicație grafică avem nevoie de un context grafic/panou. Folosind biblioteca *GLFW*, nucleul poate lucra cu 1 sau mai multe panouri. Programatorul inițializează un panou prin apelarea funcției:

```
WindowID window_create(WindowSpecification*);
```

Specificația unui panou poate avea următoarele atribute:

```
typedef struct WindowSpecification {  
    i32      width;  
    i32      height;  
    char*     name;  
  
    bool      is_fullscreen;  
    bool      is_resizable;  
    bool      has_vsync;  
    bool      has_msaa;  
  
    u32      buffer_clear_color;  
} WindowSpecification;
```

Spre exemplu, inițializarea unui panou folosind această bibliotecă:

```
window_create(&(WindowSpecification) {  
    width = 800,  
    height = 600,  
    is_resizable = true,  
    has_msaa = true,  
    has_vsync = true  
});
```

Dacă nu este precizat un nume, nucleul inițializează automat cu numele „Placeholder Window Name”.

Un panou, pe lângă informații despre sine, memorează și o listă de layere, astfel încât funcționalitatea și comportamentul unui panou este izolat de celelalte.

Nucleul poate lucra cu mai multe panouri, dar, deoarece majoritatea aplicațiilor au nevoie de un singur panou, există un macro `ENN_MULTI_WINDOW` ce poate fi definit dacă aplicația funcționează cu mai multe panouri, în caz contrar, nucleul este optimizat pentru a lucra cu unul singur.

Pentru a asocia un layer unui panou se folosește funcția:

```
LayerID window_push_layer(Layer*);
```


Spre exemplu:

```
window_push_layer(&(Layer) {
    .active = true,
    .priority = 0,
    .init = example_init,
    .term = example_term,
    .on_render = example_on_render,
    .on_update = example_on_update,
    .on_event = example_on_event
});
```

Sau pentru simplitate, sunt definite 2 macrouri:

```
LAYER_DEFINE(example);
window_push_layer(&(Layer) {
    .active = true,
    .priority = 0,
    LAYER_ASSIGN(example)
});
```

Actualizarea layerelor se face folosind id-ul primit:

```
void layer_set_active(LayerID id);
void layer_set_inactive(LayerID id);
```

Toate panourile sunt automat distruse la încetarea aplicației. Programatorul specifică doar atributele inițiale, iar mai apoi poate configura panourile folosind funcțiile:

```
void window_create(WindowSpecification* specification);
void window_destroy(void);

char* window_get_name(void);
i32vec2 window_get_framebuff_dim(void);
i32 window_get_framebuff_width(void);
i32 window_get_framebuff_height(void);
bool window_is_fullscreen(void);
bool window_has_vsync(void);

LayerID window_push_layer(Layer* layer);
void layer_set_active(LayerID id);
void layer_set_inactive(LayerID id);

void window_set_fullscreen(bool state);
void window_set_vsync(bool state);
void window_set_buffer_clear_color(u32 color_hex);

void window_flip_fullscreen(void);
void window_flip_vsync(void);

void window_make_context(void);
void window_set_viewport(i32 x, i32 y, i32 width, i32 height);
```

Nucleul nu implementează un sistem complet de rendering⁸. În schimb, acesta oferă utilitare pentru a ușura munca programatorului. Aceste utilitare sunt:

```
ShaderID render_shader_compile(const char* vert, const char* frag);
```

Prin care compilează un shader⁹ OpenGL, `vert` și `frag` sunt numele fișierelor din directorul *Simulation/Assets/Shaders*, ce corespund shaderelor de tip *vertex*, respectiv *fragment*.

Nucleul definește și o structură de date pentru citirea și prelucrarea imaginilor:

```
typedef struct Image {  
    i32 width;  
    i32 height;  
    i32 channels;  
    byte* data;  
} Image;
```

Împreună cu câteva funcții de prelucrare.

```
void render_image_create(Image* img);  
void render_image_destroy(Image* img);  
  
void render_image_read_2(Image* img, const char* filepath);  
void render_image_read_3(Image* img, const char* filepath, bool  
flip_vertical);
```

Macroul `render_image_read` va detecta numărul de argumente și va apela funcția corectă pentru citirea unei imagini.

```
TextureID render_texture_create_image(Image* img,  
                                     ENN_WRAP_ALGORITHM wrap_algo,  
                                     ENN_SCALING_ALGORITHM scale_algo);  
TextureID render_texture_create_file(const char* filename,  
                                     ENN_WRAP_ALGORITHM wrap_algo,  
                                     ENN_SCALING_ALGORITHM scale_algo);  
void render_texture_destroy(TextureID id);
```

Aceste funcții sunt folosite pentru a crea texture și a le transmite plăcii video.

⁸ Rendering = procesul de generare a imaginilor din anumite date (ex. puncte în spațiu ce definesc un corp geometric)

⁹ Shader = program ce rulează strict pe placa video, în acest proiect folosim shadere OpenGL ce sunt scrise în limbajul GLSL (OpenGL Shading Language)

II.1.3. Configurare

Pe parcursul acestei secțiuni au fost menționate macroui de configurare, acestea se află în fișierul *config.h* ce este inclus automat de nucleu, și permit schimbarea nucleului pentru aplicația dezvoltată. Configurările posibile sunt:

```
/* ----- Base Configs ----- */

#define ENN_DEBUG_MODE

#define ENN_APP_DIRECTORY           "Simulation"
#define ENN_DATA_PATH               "/Data/"
#define ENN_LOGS_PATH               "/Data/Logs/"

#define ENN_GENERAL_LOG_FILENAME_FORMAT "%Y-%m-%d-%H-%M-%S.log"
#define ENN_MEMORY_LOG_FILENAME_FORMAT "%Y-%m-%d-%H-%M-%S.memlog"
#define ENN_LOG_TIME_FORMAT          "[%H:%M:%S]: "

#define ENN_MAX_LOG_BUFF_SIZE       2048
#define ENN_MAX_CALLSTACK_SIZE      64

#define ENN_VECTOR_IMPL
#define ENN_DEQUE_IMPL

#define ENN_VECTOR_INIT_CAPACITY     8
#define ENN_DEQUE_INIT_CAPACITY     16

#define ENN_DATAFILE_KEYPATH_SEPARATOR '|'
#define ENN_DATAFILE_OPERATOR_ASSIGN '='
#define ENN_DATAFILE_STRING_IDENTIFIER '"'
#define ENN_DATAFILE_OPERATOR_LIST_BEGIN '['
#define ENN_DATAFILE_OPERATOR_LIST_END   ']'
#define ENN_DATAFILE_OPERATOR_LIST_CONTINUE ','

#define ENN_DATAFILE_LIST_ITEM_PADDING 4
#define ENN_DATAFILE_FILE_EXTENSION    ".enn"
#define ENN_DATAFILE_WHITESPACE        "\n\t"
#define ENN_DATAFILE_NUMBER_SYMBOLS    "0123456789.-"
#define ENN_DATAFILE_MAX_STRING_SIZE   64

/* ----- Optional Configs ----- */

#define ENN_INCLUDE_OPTIONAL
#define ENN_MULTI_WINDOW
#define ENN_FOCUS_WINDOW_OPTIMIZATION

#define ENN_WINDOW_MIN_WIDTH          320
#define ENN_WINDOW_MIN_HEIGHT         180

#define ENN_USE_CUSTOM_RENDERING
#define ENN_DEFAULT_RENDERER_VERTEX_BUFF_SIZE 2048

#define ENN_USER_DEFINED_EVENTS
#define ENN_ASSETS_PATH                "/Assets/"
#define ENN_SHADERS_PATH                "/Assets/Shaders/"
#define ENN_TEXTURES_PATH               "/Assets/Textures/"
#define ENN_FONTS_PATH                  "/Assets/Fonts/"
```

II.2. APLICAȚIE

II.2.1. Sistemul de rendering

Datorită graficii relativ simple necesare unui astfel de proiect, am ales să folosim un sistem de Batch Rendering. Vom inițializa un buffer global, ce memorează toate punctele ce definesc forme geometrice. Singurele forme folosite pentru acest proiect sunt: dreptunghiul, segmentul și secvențele text. La finalul fiecărui cadru vom trimite bufferul către placa video, ce, folosind shaderele compilate, va transforma datele primite în imagini concrete. Deoarece reducem transferul de date dintre procesor și placă video, acest sistem este extrem de rapid din punct de vedere al timpului pentru generare a cadrelor.

Secvențele text sunt împărțite într-o succesiune de caractere ASCII, ce sunt desenate printr-un dreptunghi texturat cu o imagine, reprezentând caracterul în sine. Această tehnică de desenare a textului poartă numele de Bitmap Font și este regăsită în majoritatea aplicațiilor vechi. Textura ce definește acest font pentru aplicația noastră se regăsește în directorul Simulation/Assets/Textures.

II.2.2. Sistemul de coordonate

Una dintre cele mai dificile etape ale acestui proiect a fost sistemul de coordonate utilizat. OpenGL folosește un sistem de coordonate numit NDC (Normalized Device Coordinates), ce diferă de majoritatea sistemelor întâlnite până acum în grafica calculatoarelor. În locul unui sistem clasic, unde 0,0 se află în partea de stânga-sus a panoului/ecranului, iar colțul dreapta-jos reprezintă lungimea și lățimea panoului, OpenGL definește centrul ecranului drept 0,0 și folosește sistemul de coordonate carteziene pentru a se referi la un punct pe ecran.

În plus, deoarece dorim ca utilizatorul să poată „mișca” editorul sau să poată efectua operații de tip zoom, trebuie să existe sisteme de coordonate diferite pentru obiectele din aplicație și obiectele afișate pe ecran.

Deoarece GLSL admite matrici și vectori matematici, iar fiecare punct pe ecran este definit drept un vector de 4 valori (x, y, z, w), toate coordonatele pentru obiectele din aplicație pot fi transformate în coordonate pentru obiecte pe ecran printr-o singură operație de înmulțire a matricilor.

Definim următoarele date despre harta internă a aplicației:

- Visible Tiles = Numărul de unități din hartă vizibile pe ecran (x_v, y_v)
- Camera Offset = Poziția „camerei” pe hartă (x_c, y_c)
- Zoom = Nivelul de zoom sau apropierea „camerei” de hartă (z)

Cu aceste 3 date putem forma o matrice de transformare ortografică pentru coordonatele de hartă în coordonate NDC.

$$\begin{pmatrix} \frac{2z}{x_v} & 0 & 0 & -\frac{2x_c z}{x_v} - 1 \\ 0 & -\frac{2z}{y_v} & 0 & \frac{2y_c z}{y_v} + 1 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} x \\ y \\ 0 \\ 1 \end{pmatrix}$$

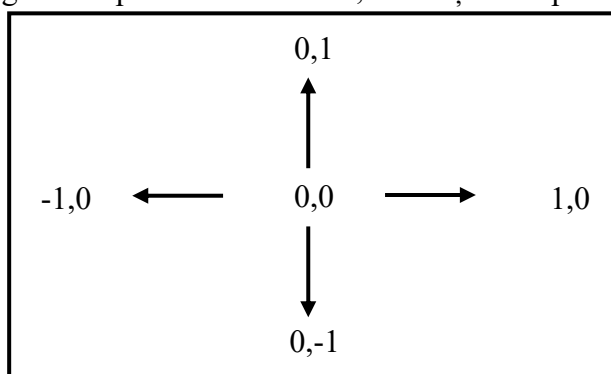


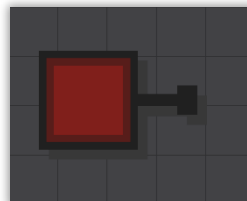
Figura II.2.2.1. Sistemul de coordonate NDC

II.2.3. Componentele simulatorului

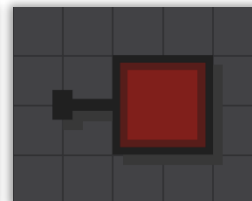
Partea centrală a proiectului este simulatorul în sine, acesta este compus din mai multe elemente la care are acces utilizatorul. Aceste elemente sunt:

Indicatorul electric

Acest element se subîmparte în indicatori de intrare și indicatori de ieșire. Aceștia sunt folosiți pentru a vizualiza starea curentului în urma unor operații sau pentru a seta starea curentului. Aceștia prezintă un singur pin ce poate fi conectat prin mai mulți conectori.



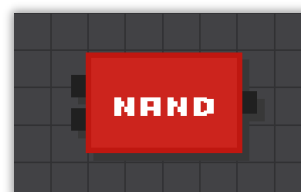
Imagine II.2.3.1
Indicator intrare



Imagine II.2.3.2
Indicator ieșire

Circuit integrat

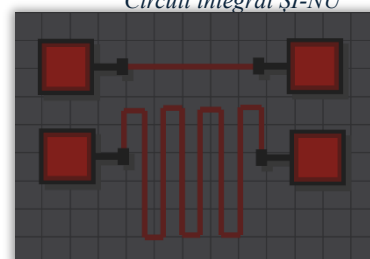
Circuitul integral sau Chipul este componenta ce poate efectua logica digitală. Utilizatorul are acces, la început, doar la un chip ce implementează operația logică ȘI-NU (NAND). Toate celelalte operații logice pot fi implementate folosind combinații ale acesteia.



Imagine II.2.3.4
Circuit integrat ȘI-NU

Conectori

Conectorii sunt o abstractizare a micilor elemente din circuitele reale ce asigură comunicarea între elemente. Aceștia sunt accesați prin apăsarea click-stângă pe un pin și apăsarea click-stângă pe un pin de tip diferit (intrare/ieșire). Culoarea conectorului depinde de culoarea elementului din care face parte primul pin apăsător. Conectorii permit „ancorarea” în puncte date de utilizator prin apăsarea click-stângă în timpul conectării a 2 pini.



Imagine II.2.3.5.
Conectori fără ancore (sus)
Conectori cu ancore (jos)

II.2.4. Logica simulatorului

Simulatorul funcționează pe un sistem de ticks. Fiecare $\frac{1}{20}$ secunde (1 tick), o funcție de execuție este apelată ce transferă starea curentului dintr-un pin de ieșire într-un pin de intrare legați prin conectori. Apoi fiecare circuit integrat execută logica internă, în mod recursiv, dacă este format din alte circuite integrate. Starea fiecărui pin este memorată de 2 ori, înainte și după executarea circuitului integrat. După executarea circuitelor, starea pinilor este copiată și execuția se termină. Memorarea dublă a stării pinilor asigură executarea corectă a circuitelor recursive.

III. DETALII TEHNICE

III.1. COMPILARE

Proiectul se folosește de produse dezvoltate de GNU¹⁰, anume GCC, AR și MAKE. Aceste utilitare sunt preinstalate pe sistemele din familia GNU/Linux, pentru sistemele Windows recomandăm instalarea pachetelor:

- Git (<https://git-scm.com/install/windows>)
- MingW64 (<https://www.mingw-w64.org/>)
- Make (<https://gnuwin32.sourceforge.net/packages/make.htm>)

Directorul Dev, conține un Makefile ce definește 3 reguli.

```
make build_debug
```

Ce compilează toate fișierele împreună cu nucleul și generează un singur executabil. Acest mod dezactivează toate optimizările și este folosit în timpul dezvoltării aplicației. Opțiunile de compilare pentru acest mod sunt:

```
-Wall -Werror -Wpedantic -std=gnu99 -O0
```

```
make build_release
```

Ce compilează fișierele într-un singur executabil pentru lansarea aplicației. Acest mod activează toate optimizările și dezactivează terminalul. Opțiunile de compilare sunt:

```
-Wall -Werror -Wpedantic -std=gnu99 -O3 -static -mwindows
```

Nucleul este compilat separat, fiecare fișier *.c* este compilat într-un fișier *.o*, ce sunt mai apoi arhivate într-un singur fișier *core.a*, ce reprezintă biblioteca nucleului. Această bibliotecă este mai apoi compilată static cu codul specific aplicației.

```
make clean
```

Ce șterge fiecare fișier *.o* și biblioteca statică pentru a recompila întreaga aplicație.

¹⁰ GNU (GNU's Not Unix) = proiect fondat de Richard Stallman ce dezvoltă produse soft gratuite și Open-Source

IV. UTILIZAREA APLICAȚIEI

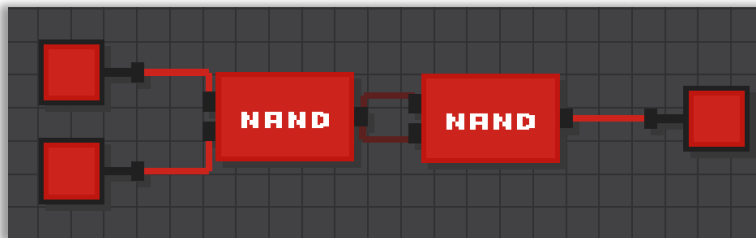
Aplicația finală prezintă o interfață grafică ușor de înțeles și încurajează explorarea circuitelor și comportamentul intern al electronicelor prin logică binară. În continuare vom prezenta exemple pentru implementarea unor operații logice și unități rudimentare de logică matematică.

Operația NOT



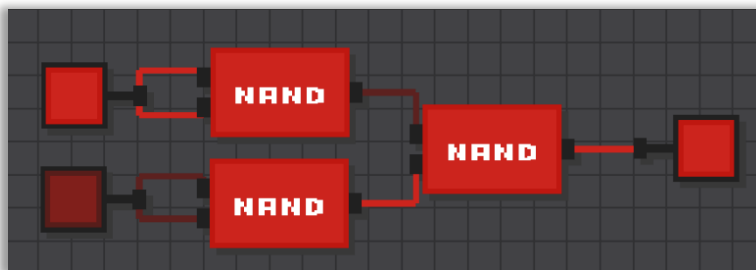
INPUT		OUTPUT
1		0
0		1

Operația AND



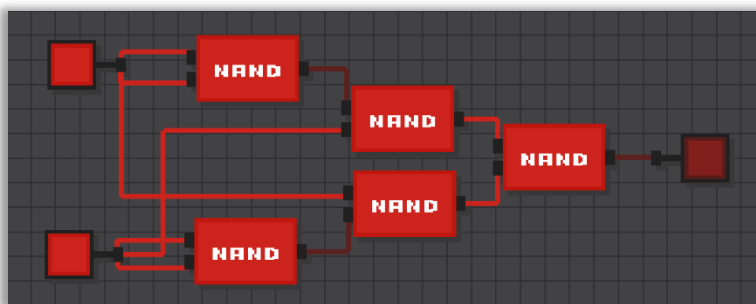
INPUT		OUTPUT
0	0	0
0	1	0
1	0	0
1	1	1

Operația OR



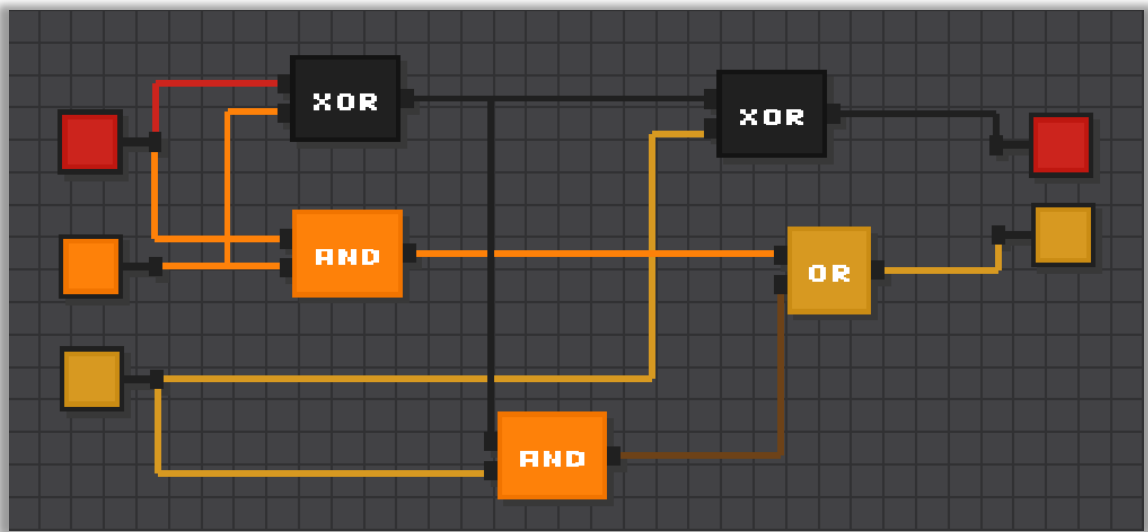
INPUT		OUTPUT
0	0	0
0	1	1
1	0	1
1	1	1

Operația XOR



INPUT		OUTPUT
0	0	0
0	1	1
1	0	1
1	1	0

Adunare a 2 biți



INPUT			OUTPUT	
A	B	Carry	Sum	Carry
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

V. BIBLIOGRAFIE

Joey de Vries, *Learn OpenGL – Graphics Programming*, Kendall & Welling, June 2020

Sebastian Lague, *Exploring How Computers Work*, 16 Nov. 2020,
<https://www.youtube.com/watch?v=QZwneRb-zqA>

Sebastian Lague, *How Do Computers Remember?*, 4 Dec. 2020,
<https://www.youtube.com/watch?v=I0-izyq6q5s>

Eskil Steenberg, *How I Program C*, 21 Nov. 2016,
<https://www.youtube.com/watch?v=443UNeGrFoM&>

Eskil Steenberg, *Debugging and the art of avoiding bugs*, 24 Aug. 2025,
<https://www.youtube.com/watch?v=sfrnU3-EpPI>

Muukid, *Creating a window, Software from Scratch*, 28 Sept. 2024,
https://www.youtube.com/watch?v=jG1_nZ7V0wE&

Muukid, *Rendering 2D objects, Software from Scratch*, 7 Dec. 2024,
https://www.youtube.com/watch?v=Lsh_0dz7p4U&

Javidx9, *Programming Panning & Zooming*, 10 Jun. 2018,
<https://www.youtube.com/watch?v=ZQ8qtAizis4>

Javidx9, *Serialising/Deserialising Structured Memory To Disk, AKA Savin' Stuff*, 19 Nov. 2022, <https://www.youtube.com/watch?v=jlS1Y2-yKV0&>

VoxelRifts, *A Quick, Easy and Extendable OpenGL Renderer in 250 LOC!*, 17 Feb. 2023,
<https://www.youtube.com/watch?v=NPnQF4yABwg>

Yan Chernikov, *How to Build a 2D Renderer, Game Engine Series*, 15 Sept. 2019,
<https://www.youtube.com/watch?v=ULO8JhM9FsQ>

Yan Chernikov, *Batch Rendering*, 22 Jan. 2020,
<https://www.youtube.com/watch?v=Th4huqR77rI>

Yan Chernikov, *C++ Application Architecture*, 17 Sept. 2025,
<https://www.youtube.com/watch?v=rUxZ5N77M5E>